PLAN DE ACCIÓN Y CONSENSO TÉCNICO • AUDITORÍA POST-HITO 2

Plan de Remediación Integral y Hoja de Ruta de Implementación

Respuesta detallada y acuerdos de diseño frente a la retroalimentación de Samir: eliminación nativa de SQL dinámico, estrategia no disruptiva de contraseñas, desacople E2E y matriz de cierre verificable.

PROYECTO
Librería y Papelería Mundo Escolar (SQL Server 2022 / NestJS / React)

INTERLOCUTORES TÉCNICOS
Equipo de Desarrollo (Mau) & Auditoría Técnica (Samir)

DOCUMENTO PREVIO
Dictamen Técnico de Respuesta a Auditoría (Calificación Samir: 10/10 Análisis)

OBJETIVO DEL DOCUMENTO
Consensuar el diseño exacto de código antes de iniciar la remediación física

ESTADO DE LA PROPUESTA
Aceptación Total de Recomendaciones • Hoja de Ruta P0/P1 Aprobada

FECHA DE EMISIÓN
Noviembre de 2026

Consenso Técnico y Declaración Metodológica

El equipo de desarrollo de Mundo Escolar recibe y adopta en su totalidad las observaciones de Samir. Coincidimos plenamente en su evaluación: **haber identificado y reconocido los riesgos con un 10/10 no equivale a haberlos resuelto (0/10 en implementación demostrada)**. Por tanto, antes de precipitar cambios en el repositorio o declarar victorias prematuras ante el profesor Torrentes, este documento formaliza los detalles de ingeniería de cada solución (incluyendo la excelente propuesta de sintaxis nativa T-SQL de Samir, el plan de migración de contraseñas con prefijos de versión y el desacople de pruebas E2E), definiendo con exactitud qué se codificará, cómo se probará y qué evidencias se aportarán para el Hito #3.

1. Adopción de la Sintaxis Nativa T-SQL: Eliminación Definitiva de SQL Dinámico

La recomendación técnica aportada por Samir en la Sección 2 de su informe representa una mejora de diseño sustancial. En lugar de intentar sanear cadenas complejas dentro de `sp_executesql`, Microsoft SQL Server admite el paso de variables T-SQL directamente en las sentencias de respaldo y verificación física.

> **Aporte Técnico de Samir:** En T-SQL, las instrucciones `RESTORE VERIFYONLY FROM DISK = @Variable` y `BACKUP DATABASE ... TO DISK = @Variable` son construcciones nativas del motor. **No requieren SQL dinámico ni concatenación de cadenas**. Al no existir concatenación, el analizador sintáctico de SQL Server trata el parámetro estrictamente como un literal de dispositivo de almacenamiento, extirpando cualquier posibilidad de inyección SQL a nivel de gramática.

1.1. Rediseño del Procedimiento SP_DBA_VerificarIntegridadBackup

El procedimiento se reescribirá en `sql/01_despliegue_secuencial/10_CrearProcedimientos.sql` abandonando `sp_executesql` y validando canónicamente la ruta:

```
-- Procedimiento 72 Rediseñado (Sin SQL Dinámico)
CREATE OR ALTER PROCEDURE dbo.SP_DBA_VerificarIntegridadBackup
    @NombreArchivoBak VARCHAR(255)  -- Solo recibe el nombre del archivo (ej. 'MUNDO_ESCOLAR_FULL_....bak')
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        -- 1. Validación estricta de seguridad: Rechazar traversals y rutas de red UNC
        IF @NombreArchivoBak LIKE '%..%' OR @NombreArchivoBak LIKE '%\\%' OR @NombreArchivoBak LIKE '%/%' OR @NombreArchivoBak LIKE '
        BEGIN
            THROW 50001, 'Nombre de archivo no válido o contiene caracteres no permitidos.', 1;
        END;

        -- 2. Confinamiento forzado al directorio canónico seguro del servidor
        DECLARE @RutaAbsoluta NVARCHAR(500) = N'C:\SQLBackup\' + @NombreArchivoBak;

        -- 3. EJECUCIÓN ESTÁTICA NATIVA (Sin sp_executesql ni concatenación)
        RESTORE VERIFYONLY FROM DISK = @RutaAbsoluta;

        INSERT INTO Bitacora (Id_Usuario, Accion, Tabla_Afectada, Id_Registro_Afectado, Fecha_Hora, Descripcion)
        VALUES (1, 'VERIFY_BACKUP_OK', 'DATABASE', 0, GETDATE(), LEFT(CONCAT('Verificado: ', @NombreArchivoBak), 200));

        SELECT 'El archivo de respaldo es íntegro y legible.' AS Resultado, @NombreArchivoBak AS Archivo;
    END TRY
    BEGIN CATCH
        THROW;
    END CATCH
END;
```

1.2. Rediseño de SP_DBA_EjecutarBackupCompleto, Diferencial y Log

De igual manera, los procedimientos 69, 70 y 71 adoptarán la sintaxis estática:

```
-- Ejecución nativa sin sp_executesql
DECLARE @RutaDestino NVARCHAR(500) = N'C:\SQLBackup\MUNDO_ESCOLAR_FULL_' + @FechaStr + N'.bak';
BACKUP DATABASE MUNDO_ESCOLAR TO DISK = @RutaDestino WITH FORMAT, INIT, COMPRESSION, STATS = 10;
```

1.3. Desactivación de TRUSTWORTHY y Matiz del Exploit

- **Desactivación Inmediata:** Se ejecutará `ALTER DATABASE MUNDO_ESCOLAR SET TRUSTWORTHY OFF;` en los scripts secuenciales y de mantenimiento.
- **Aceptación del Matiz de Samir:** En el informe final de Hito 3, no calificaremos la vulnerabilidad previa como "RCE público e incondicional", sino con la precisión técnica que Samir indicó: "*Falla de defensa en profundidad con riesgo de elevación interna de privilegios en componentes administrativos protegidos por JWT*".

2. Plan de Migración de Contraseñas No Disruptivo (Zero-Downtime Migration)

En la Observación B de su revisión, Samir realizó una advertencia de arquitectura indispensable: "*Migrar las contraseñas requiere más que cambiar una función... Mundo Escolar utiliza procedimientos almacenados y funciones de autenticación que podrían depender del formato actual de los hashes*".

A continuación se formaliza la arquitectura de migración paso a paso para garantizar que **ningún usuario existente pierda acceso y la transición ocurra de forma transparente**:

2.1. Inspección de la Capa de Datos en SQL Server

Se revisaron los contratos de los procedimientos almacenados en `10_CrearProcedimientos.sql`:

- `SP_Usuario_Crear`: Recibe `@ContrasenaHash VARCHAR(255)` y la inserta en `dbo.Usuario`. SQL Server **no calcula el hash**; solo almacena la cadena provista por el backend.
- `SP_Usuario_ObtenerParaAutenticacion`: Devuelve la columna `ContrasenaHash VARCHAR(255)` hacia el backend.
- La columna en `03_CrearTablas.sql` está definida como `ContrasenaHash VARCHAR(255) NOT NULL`. Dado que un hash Argon2id mide típicamente ~96-128 caracteres y un hash bcrypt mide 60 caracteres, **la columna actual de 255 caracteres tiene capacidad nativa de almacenar los nuevos hashes sin alterar el esquema relacional**.

2.2. Algoritmo de Verificación Dual y Actualización Dinámica (In-Flight Rehash)

El backend en `backend/src/auth/auth.service.ts` y `password.util.ts` implementará un validador discriminado por prefijo:

```
// Lógica de Verificación y Auto-Actualización en auth.service.ts
async function validarYMigrarCredenciales(usuario, contrasenaPlana) {
  const hashBD = usuario.ContrasenaHash;
  let esValida = false;
  let requiereRehash = false;

  // Caso 1: Usuario ya migrado con Argon2id / bcrypt (Inicia con prefijo estándar)
  if (hashBD.startsWith('$argon2id$') || hashBD.startsWith('$2b$')) {
    esValida = await verificarHashLento(contrasenaPlana, hashBD);
  }
  // Caso 2: Usuario legado con SHA-256 (Cadena hexadecimal de 64 caracteres)
  else if (/^[A-Fa-f0-9]{64}$/.test(hashBD)) {
    esValida = verificarSha256Legado(contrasenaPlana, hashBD);
    if (esValida) {
      requiereRehash = true; // Contraseña comprobada: lista para upgrade inmediato
    }
  }

  if (!esValida) return null;

  // Auto-migración transparente en el momento del login exitoso
  if (requiereRehash) {
    const nuevoHash = await generarNuevoHashLento(contrasenaPlana);
    await actualizarHashEnBD(usuario.Id_Usuario, nuevoHash);
    logger.log(`Usuario ${usuario.Id_Usuario} migrado con éxito a hash moderno.`);
  }

  return usuario;
}
```

2.3. Beneficios Técnicos del Esquema Propuesto

1. **Cero Pérdida de Acceso:** Los usuarios administrativos (`admin`, vendedores, bodegueros, clientes) ingresan normalmente con sus credenciales actuales.
2. **Actualización Automática Silenciosa:** En el milisegundo en que el usuario digita su clave correcta, el backend la re-encripta con el nuevo algoritmo seguro con sal individual y guarda el nuevo hash.
3. **Compatibilidad Total con Clientes Nuevos:** Todo nuevo cliente registrado mediante `POST /clientes/*` se almacena directamente con el algoritmo seguro moderno.

3. Matriz Integral de Cierre para los 10 Hallazgos (Atención Obs. A)

Atendiendo la Observación A de Samir, se expande la matriz técnica para que contemple **las diez observaciones identificadas originalmente**, incorporando los compromisos explícitos que habían quedado fuera de la versión anterior:

ID	VULNERABILIDAD	ACCIÓN TÉCNICA CONCRETA	ARCHIVO / MÓDULO	CRITERIO DE ACEPTACIÓN
S01	Clave predeterminada en clientes	Hacer obligatorio `contrasena` en DTOs (mín. 8 car., números, letras). Eliminar fallback `'Cliente2026!'`.	`clientes.service.ts` `clientes.dto.ts`	Petición sin clave rechazada con HTTP 400.
S02	Credenciales en Git (Obs. E)	Rotar contraseñas en instancias locales. Parametrizar `Seguridad.sql` con placeholders seguros `$(PASS_ADMIN)`.	`Seguridad.sql`	Cero contraseñas en texto claro en el historial nuevo.
S03	Hash SHA-256 sin sal (Obs. B)	Implementar verificación dual y upgrade en caliente (In-flight rehash) hacia bcrypt/Argon2id.	`password.util.ts` `auth.service.ts`	Usuarios legados inician sesión y se auto-migran.
S04	TRUSTWORTHY ON y sa	Ejecutar `ALTER DATABASE ... SET TRUSTWORTHY OFF;`. Eliminar `EXECUTE AS OWNER`.	`01_CrearBaseDatos.sql` `10_CrearProc.sql`	`is_trustworthy_on = 0` en `sys.databases`.
S05	Privilegios amplios en app	Retirar `db_backupoperator` de `usr_mundo_app`. Revocar `GRANT EXECUTE ON SCHEMA::dbo`; conceder por SP.	`Seguridad.sql`	Login de app no puede hacer BACKUP directo.
S06	Conexión SQL sin cifrado	Eliminar fallback a `sa` en `configuration.ts`. Requerir variables `DB_USER` y `DB_PASSWORD`.	`configuration.ts`	Backend no inicia si faltan credenciales explícitas.
S07	Falta de Rate Limiting	Instalar `@nestjs/throttler`. Configurar límite de 10 req/min en login y registro.	`app.module.ts` `auth.controller.ts`	HTTP 429 tras 10 intentos consecutivos fallidos.
S08	JWT en localStorage (Obs. D)	Reducir expiración de JWT a 1 hora. Instalar `helmet` con directivas CSP estrictas. Documentar riesgo residual.	`main.ts` `configuration.ts`	Cabeceras CSP activas y TTL de token acotado.
S09	Divulgación de errores SQL	Modificar `SqlExceptionFilter` para devolver mensajes de negocio opacos y registrar el detalle en logs internos.	`sql-exception.filter.ts`	Respuestas HTTP no revelan nombres de tablas ni columnas.
S10	Swagger público	Habilitar `/api/docs` únicamente si `NODE_ENV !== 'production'`. Sanitizar `/health`.	`main.ts` `sql.service.ts`	Swagger inaccesible en modo producción.
AUD	Vulnerabilidades npm audit	Ejecutar `npm audit fix` en backend y frontend para resolver `multer`, `body-parser` y `msal-node`.	`backend/package.json`	Cero vulnerabilidades altas en dependencias de prod.

3.1. Corrección Definitiva de Pruebas E2E (Atención Obs. C)

Para solucionar de raíz la falla identificada por Samir en `backend/test/app.e2e-spec.ts`, se aplicará un desacople en dos capas:

1. **Soporte a Variables de Entorno Dinámicas:** El `MockConfigService` de la prueba leerá primero `process.env.DB_PASSWORD` y `process.env.DB_USER`. De este modo, en la máquina de Samir, en la de Mau o en un pipeline CI, la prueba usará las credenciales locales de quien la ejecute.
2. **Aislamiento de Pruebas Unitarias de Controladores:** Se incorporará una suite unitaria pura con mock de `SqlService` para verificar el flujo HTTP sin requerir que SQL Server esté corriendo.

3.2. Tratamiento Formal de JWT y Riesgo Residual (Atención Obs. D)

> **Justificación Técnica del Almacenamiento Local:** Se decide mantener `localStorage` para la persistencia del token en esta fase debido a la naturaleza desacoplada de la SPA en React (servida en puerto 5174) frente al backend NestJS (puerto 3000), lo que evita problemas de bloqueo de cookies cross-site en navegadores modernos durante pruebas locales.
>
> **Riesgo Residual Aceptado:** Si existiera una inyección de script malicioso (XSS), el token podría ser leído.
> **Controles Compensatorios Obligatorios:** 1) Reducción del ciclo de vida del token a 60 minutos; 2) Implementación de `helmet` con Content-

> Security-Policy (CSP) para impedir la ejecución de scripts no autorizados; 3) React escapa de forma nativa las cadenas renderizadas en JSX, neutralizando XSS reflejado.

4. Protocolo de Pruebas de Respaldos para el Banco de Evidencias

Atendiendo la recomendación de Samir en la Sección 4 de su informe (*"RESTORE VERIFYONLY no equivale a restaurar exitosamente toda una base de datos. Una prueba real de recuperación proporciona evidencia más sólida"*), se acuerda generar el siguiente conjunto de evidencias verificables una vez codificados los cambios:

PRUEBA / EVIDENCIA	PROCEDIMIENTO TÉCNICO A EJECUTAR	RESULTADO COMPROBABLE
1. Respaldo Completo (Full)	Ejecución de `dbo.SP_DBA_EjecutarBackupCompleto` mediante botón web o comando T-SQL directo.	Archivo `.bak` generado en `C:SQLBackup`, compresión nativa verificada (~89%) y fila en `dbo.Bitacora`.
2. Respaldo Diferencial (Diff)	Inserción de registros de prueba y ejecución de `dbo.SP_DBA_EjecutarBackupDiferencial`.	Archivo `DIFF.bak` generado con tamaño reducido correspondiente solo a los cambios recientes.
3. Respaldo de Log (.trn)	Ejecución de `dbo.SP_DBA_EjecutarBackupLog` (en modelo FULL).	Archivo `.trn` generado y truncado del log virtual de transacciones.
4. Verificación Física Nativa	Llamada a `dbo.SP_DBA_VerificarIntegridadBackup` con la nueva sintaxis sin concatenación.	Resultado `RESTORE VERIFYONLY` exitoso reportado en UI y en bitácora.
5. Restauración Real DRP	Ejecución del script maestro `03_Prueba_Recuperacion_DRP.sql` restaurando en `MUNDO_ESCOLAR_DRP_TEST`.	Restauración de 2,618 páginas en disco secundario, `DBCC CHECKDB` limpio (0 errores) y desmantelamiento seguro.
6. Prueba Negativa de Inyección	Intento deliberado de enviar caracteres de escape (`' OR 1=1; --`) y rutas UNC (`\atacanteshare`).	Rechazo inmediato por el validador con error 50001, sin ejecución en el motor.

5. Adopción de la Redacción de Samir para el Informe del Hito #3

El equipo de desarrollo adopta de manera íntegra y sin modificaciones la redacción propuesta por Samir en la Sección 5 de su reporte para documentar este apartado en el informe final que se presentará al profesor Torrentes:

> **Texto Oficial Aprobado para el Hito #3:**
>
> *"Posterior a la implementación de los módulos funcionales y administrativos del sistema Mundo Escolar, se realizó una revisión técnica de la seguridad del backend, la configuración de SQL Server y los mecanismos de mantenimiento de la base de datos.*
>
> *Esta evaluación permitió identificar debilidades relacionadas con la gestión de credenciales, el almacenamiento de contraseñas, la asignación de privilegios, la protección de los endpoints y la ejecución de procedimientos almacenados administrativos.*
>
> *Como resultado, se elaboró un dictamen técnico que analiza las causas de los hallazgos, reconoce los riesgos detectados y establece un plan de remediación priorizado. Dicho plan contempla medidas de protección para la autenticación, las conexiones SQL, los respaldos y la administración de permisos, cuya implementación deberá verificarse mediante pruebas y evidencias técnicas."*

Esta formulación es **estratégicamente inexpugnable** ante el profesor, ya que transforma lo que pudo haber sido una llamada de atención por debilidades no detectadas en una demostración palpable de madurez de ingeniería de software y control de calidad interno.

6. Cronograma de Implementación y Siguientes Pasos

Con el fin de mantener un control estricto de versiones y evitar desestabilizar la rama principal, el trabajo de desarrollo se organizará en tres fases secuenciales:

FASE	TAREAS TÉCNICAS A DESARROLLAR	ENTREGABLE EN REPOSITORIO	RESPONSABLE / VALIDACIÓN
Fase 1: Motor SQL	• Reescribir SPs 69 a 72 con sintaxis estática nativa. • Desactivar `TRUSTWORTHY OFF`. • Remover `db_backupoperator` de `usr_mundo_app`. • Actualizar `03_Prueba_Recuperacion_DRP.sql`.	Scripts SQL actualizados en `sql/01_despliegue_secuencial/` y `sql/03_administracion_mantenimiento/`.	Equipo Mau • Verificado mediante ejecución en SSMS.
Fase 2: Backend NestJS	• Hacer obligatoria clave en DTOs de clientes. • Implementar verificación dual y upgrade dinámico a Argon2id/bcrypt. • Reparar E2E en `app.e2e-spec.ts`. • Instalar `@nestjs/throttler` y sanitizar errores SQL. • Ejecutar `npm audit fix` en producción.	Controladores, servicios y DTOs actualizados en `backend/src/`.	Equipo Mau • Verificado mediante compilación y tests.
Fase 3: Certificación y Evidencias	• Ejecutar batería completa de unitarias y E2E portables. • Compilar frontend (Vite) y backend (NestJS) con 0 errores. • Ejecutar protocolo de respaldos y DRP en vivo. • Generar informe final de evidencias comprobadas.	Commit consolidado en `Feature-Mau` y merge limpio a `development`.	Auditoría Samir • Calificación de implementación 10/10.

7. Conclusión Final

El diálogo técnico sostenido a partir de la auditoría de Samir ha enriquecido el proyecto Mundo Escolar de forma determinante. Contar con una contraparte técnica rigurosa que revise el código con estándares de producción ha permitido corregir vicios de diseño comunes en prototipos académicos antes de su evaluación definitiva.

Con la presentación de este plan consensuado, **el equipo se declara listo para iniciar la ejecución física de las Fases 1 y 2 en el momento en que Samir dé su visto bueno final**, garantizando que el Hito #3 se entregue con la más alta calificación técnica y funcional.